

Task 54: Find if Path Exists in Graph

Problem Description

There is a bi-directional graph with n vertices, where each vertex is labeled from 0 to $n - 1$ (inclusive). The edges in the graph are represented as a 2D integer array `edges`, where each `edges[i] = [ui, vi]` denotes a bi-directional edge between vertex `ui` and vertex `vi`. Every vertex pair is connected by at most one edge, and no vertex has an edge to itself. You want to determine if there is a valid path that exists from vertex `source` to vertex `destination`. Given `edges` and the integers `n`, `source`, and `destination`, return `true` if there is a valid path from `source` to `destination`, or `false` otherwise.

Java Solution

```
class Solution {
    public boolean validPath(int n, int[][] edges, int source, int
destination) {
        if (source == destination) return true;

        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int[] edge : edges) {
            graph.get(edge[0]).add(edge[1]);
            graph.get(edge[1]).add(edge[0]);
        }

        boolean[] visited = new boolean[n];
        Queue<Integer> queue = new LinkedList<>();
        queue.offer(source);
        visited[source] = true;

        while (!queue.isEmpty()) {
            int curr = queue.poll();
            if (curr == destination) return true;

            for (int neighbor : graph.get(curr)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.offer(neighbor);
                }
            }
        }

        return false;
    }
}
```